Grafana

Complete Handwritten Notes

Monitoring • Visualization • Alerting • Real-World DevOps Use Cases

Data Sources

- Prometheus (Metrics)
- CloudWatch (AWS Metrics)
- Kubernetes (Cluster Metrics)
- Loki (Logs)
- Elasticssearch (Logs & Data)
- Database (MySQL, PostgreSQL)



Visualization

Dashboards

Alerting

Alerts & Notifications

Monitoring

Real-time Monitoring

What You Will Learn?

- Grafana fundamentals and architecture
- Installation and configuration
- Data source integration (Prometheus, CloudWatch, Loki, etc.)
- Dashboards and visualization
- Alerting and notification setup
- Kubernetes and AWS monitoring
- Real-time production use cases
- Troubleshooting common issues
- Interview questions and answers
- Real-world DevOps project

Use Cases

- Infrastructure Monitoring (Servers, EC2, EKS)
- Container & Kubernetes Monitoring (Pods, Nodes, Applications)
- AWS Monitoring (CloudWatch Metrics)
- Application Monitoring (APIs, Microservices)
- Log Visualization (Loki / Elasticsearch)
- Alerting & Incident Management (Production Support)

Grafana Introduction

Overview, Features, Architecture and Use Cases

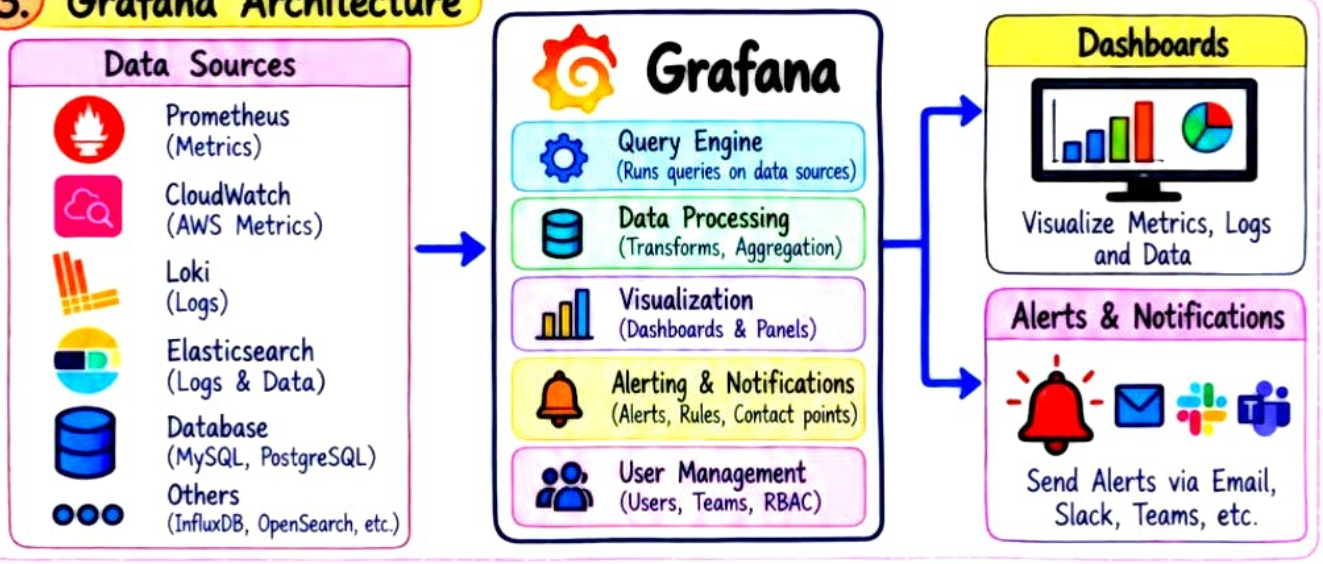
1. What is Grafana ?

- Grafana is an open-source visualization and monitoring tool used to create dashboards for metrics and logs.
- It allows us to collect data from different sources (like Prometheus, CloudWatch, Loki, Elasticsearch, databases) and visualize it in real-time.
- Grafana helps DevOps and operations teams to monitor infrastructure, applications and business metrics using beautiful dashboards, alerts and notifications.

2. Why we use Grafana ?

- Visualize metrics and logs in real-time
- Monitor infrastructure, applications and services
- Create custom dashboards for better visibility
- Integrate with multiple data sources
- Set alerts and notifications for critical issues
- Helps in faster troubleshooting and root cause analysis
- Widely used in DevOps, SRE and production support
- Supports open-source and enterprise environments

3. Grafana Architecture



4. Key Features of Grafana

- Open-source and easy to use
- Supports multiple data sources
- Customizable dashboards and panels
- Real-time metrics and log visualization
- Powerful alerting and notification system
- Supports variables, templates and annotations
- Role-based access control (RBAC)
- Plugins and extensibility
- Widely used in Kubernetes and AWS environments
- Community and enterprise support

5. Popular Data Sources

Data Source	Use Case
Prometheus	Infrastructure & application metrics
CloudWatch	AWS metrics and logs
Loki	Log aggregation
Elasticsearch	Search and visualize logs/data
InfluxDB	Time-series data
MySQL	Business/application data
PostgreSQL	Application and business data
OpenSearch	Logs and metrics

6. Use Cases of Grafana

- Monitor server health (CPU, Memory, Disk)
- Monitor Kubernetes clusters (EKS/AKS)
- Visualize application metrics (Pods, Microservices)
- Monitor AWS resources (EC2, RDS, ALB, etc.)
- View application logs (using Loki / Elasticsearch)
- Create business dashboards (User & Revenue metrics)
- Set alerts for failures, high latency, or errors
- Used in DevOps, SRE and production support

7. Grafana vs Other Monitoring Tools

Tool	Purpose	Focus Area
Grafana	Visualization	Metrics, Logs, Dashboards
Prometheus	Metrics Collection	Metrics (Time-series)
CloudWatch	AWS Monitoring	AWS Metrics & Logs
Loki	Log Aggregation	Logs (Lightweight)
Elasticsearch	Log Search & Analytics	Logs & Full-text Search
Datadog	Full Observability	Metrics, Logs, Traces, APM
Zabbix	Infrastructure Monitoring	Server & Network Monitoring
Nagios	Host & Service Monitoring	Alerts & Uptime Monitoring

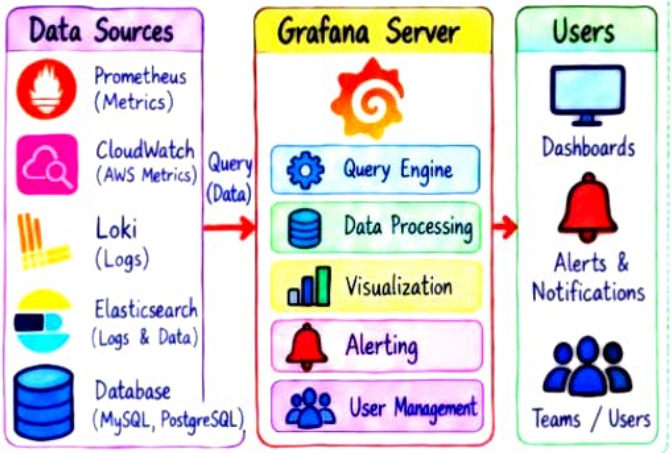
Grafana Architecture & Components

Understanding How Grafana Works and Its Key Components

1. Grafana Architecture Overview

- Grafana follows a simple architecture where it connects to one or more data sources, queries the data, processes it and visualizes it in dashboards.
- It does not store metrics by default (it stores only dashboards, users, config, etc. in its own database).
- Data is fetched from external data sources like Prometheus, CloudWatch, Loki, Elasticsearch and displayed in real-time.

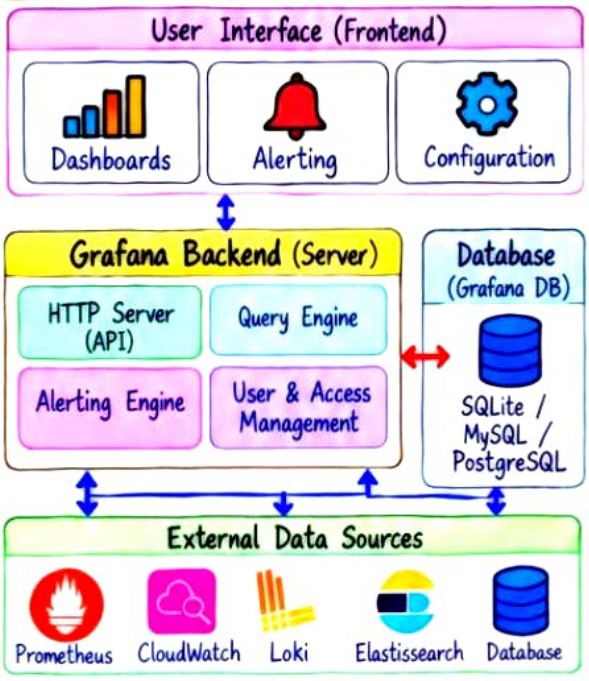
2. High Level Architecture



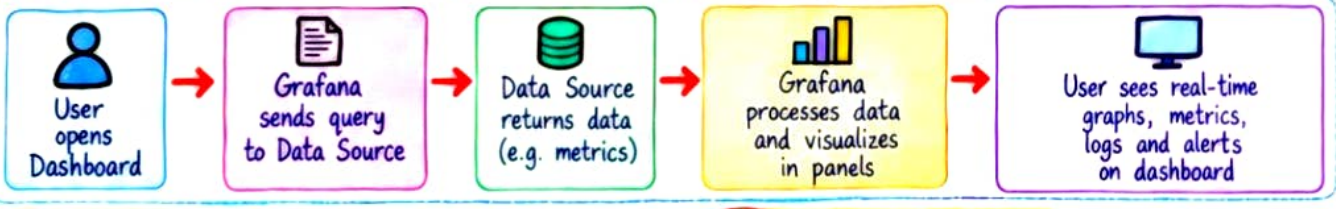
3. Key Components of Grafana

Component	Description
Data Sources	External systems from which Grafana fetches data (Prometheus, CloudWatch, etc.).
Query Engine	Responsible for querying data from data sources.
Dashboard	Collection of panels to visualize data.
Panel	Individual visualization (graph, table, gauge, etc).
Alerting	Set alert rules and send notifications.
User Management	Manage users, teams, roles and permissions.
Configuration	Datasource, dashboards, alerts and system settings.
Database (Internal)	Stores Grafana configuration, dashboards, users and alert rules (SQLite, MySQL, PostgreSQL).

4. Internal Architecture



5. Request Flow in Grafana



6. Grafana Database (Internal)

- Grafana stores its own configuration data in a database.
- By default it uses SQLite (for single instance).
- In production we use MySQL or PostgreSQL.
- It stores:
  - Users, teams and permissions
  - Dashboards
  - Alerts and notification rules
  - Data source configurations
  - Organization settings

Note:
Grafana does not store metrics data.

7. Important Points

- Grafana does not store metrics (only visualization tool).
- It can connect to multiple data sources.
- Supports real-time visualization and alerting.
- It is lightweight and easy to deploy.
- Can be self-hosted or use Grafana Cloud (managed).
- Used widely in DevOps, SRE and production environments.
- Provides powerful dashboards, alerting and extensibility.
- Role-based access control (RBAC) is available.
- Can be integrated with multiple data sources like Prometheus, CloudWatch, Loki, Elasticsearch, etc.

Grafana Installation & Configuration

Install, Configure and Access Grafana on Linux

1. Prerequisites

- Linux server (Ubuntu / Amazon Linux)
- Sudo or root access
- Internet connection
- Open ports (3000 for Grafana)
- Basic knowledge of systemd and firewall

2. Installation Steps (Ubuntu / Debian)

```
# 1. Update packages
sudo apt update -y
# 2. Install required packages
sudo apt install -y apt-transport-https software-properties-common wget
# 3. Add Grafana GPG key
wget -q -O - https://packages.grafana.com/gpg.key | \
sudo gpg --dearmor -o /usr/share/keyrings/grafana.gpg
# 4. Add Grafana repository
echo "deb [signed-by=/usr/share/keyrings/grafana.gpg] \
https://packages.grafana.com/oss/deb stable main" | \
sudo tee /etc/apt/sources.list.d/grafana.list
# 5. Install Grafana
sudo apt update -y
sudo apt install -y grafana
```

3. Installation Steps (Amazon Linux)

```
# 1. Install required packages
sudo yum install -y wget

# 2. Add Grafana repository
sudo tee /etc/yum.repos.d/grafana.repo <<EOF
[grafana]
name=Grafana
baseurl=https://packages.grafana.com/oss/rpm
enabled=1
gpgcheck=1
gpgkey=https://packages.grafana.com/gpg.key
EOF

# 3. Install Grafana
sudo yum install -y grafana

# 4. Start and enable service
sudo systemctl start grafana-server
sudo systemctl enable grafana-server
```

4. Verify Installation

```
# Check service status
sudo systemctl status grafana-server
# Check installed version
grafana-server -v
```

Expected Output

Grafana v10.x.x

5. Access Grafana in Browser

- Default URL: `http://<server-ip>:3000`
- Default Username: **admin**
- Default Password: **admin**
- Change password after first login



6. Firewall Configuration

Firewall Tool	Command
UFW (Ubuntu)	sudo ufw allow 3000/tcp
FirewallD (Amazon Linux)	sudo firewall-cmd --permanent --add-port=3000/tcp sudo firewall-cmd --reload
Security Group (AWS)	Allow inbound TCP port 3000 from your IP / required range

8. Important Configuration (grafana.ini)

Main configuration file: **/etc/grafana/grafana.ini**
Contain server settings, auth, security, database and more.

```
[server]
http_port = 3000              # Grafana server port
domain = localhost            # Domain or IP
root_url = http://localhost:3000  # Root URL

[security]
admin_user = admin            # Default admin user
admin_password = admin        # Default admin password

[users]
allow_sign_up = false         # Disable public signup
auto_assign_org = true        # Auto assign to org
auto_assign_org_role = Viewer # Default role for new users
```

9. Service Management

Start Grafana
sudo systemctl start grafana-server

Stop Grafana
sudo systemctl stop grafana-server

Restart Grafana
sudo systemctl restart grafana-server

Enable at boot
sudo systemctl enable grafana-server

Check status
sudo systemctl status grafana-server

10. Default Ports

Port	Purpose
3000	Grafana Web UI (HTTP)
443	Grafana Web UI (HTTPS) (if configured)

11. Next Actions

- Install Grafana on server
- Access the UI and login
- Change default password
- Configure data sources (Prometheus / CloudWatch)
- Create first dashboard
- Set up alerts and notifications
- Integrate with other tools

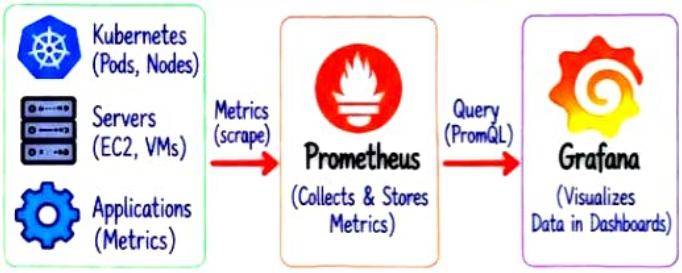
Prometheus + Grafana Integration

Integrate Prometheus with Grafana for Metrics Visualization

1. Overview

- Prometheus collects metrics from applications, servers and Kubernetes.
- Grafana uses Prometheus as a data source to visualize these metrics in dashboards.
- Prometheus stores time-series data and provides a query language called PromQL.
- Grafana fetches the data from Prometheus and displays it in the form of graphs, tables, gauges, etc.

2. Integration Architecture



3. Prometheus Configuration (prometheus.yml)

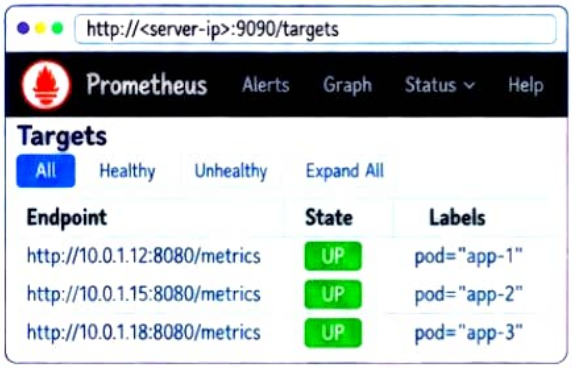
```yaml
global:
  scrape_interval: 15s     # How often to scrape
scrape_configs:
- job_name: 'kubernetes'
  kubernetes_sd_configs:
    - role: pod
  relabel_configs:
   - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
     action: keep
     regex: true
   - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
     action: replace
     target_label: __metrics_path__
     regex: (.+)
   - source_labels: [__address__,__meta_kubernetes_pod_annotation_prometheus_io_port]
     action: replace
     target_label: __address__
     rerget: (.+):(\d+)
     replacement: $1:$2
```

4. Start Prometheus

```bash
# Start Prometheus (if installed locally)
sudo systemctl start prometheus
sudo systemctl enable prometheus

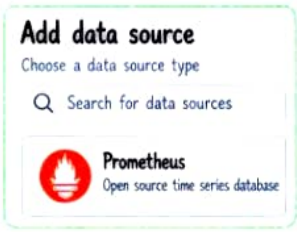
# Check status
sudo systemctl status prometheus
```

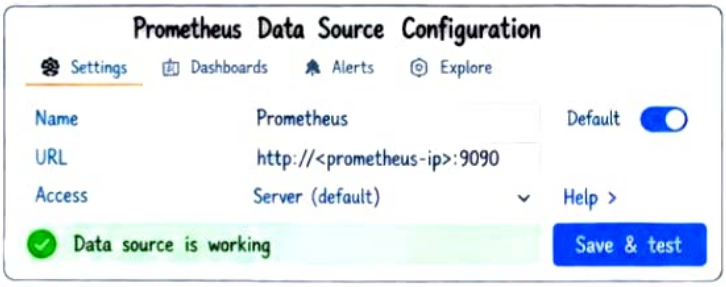
5. Verify Prometheus Metrics



6. Add Prometheus as Data Source in Grafana

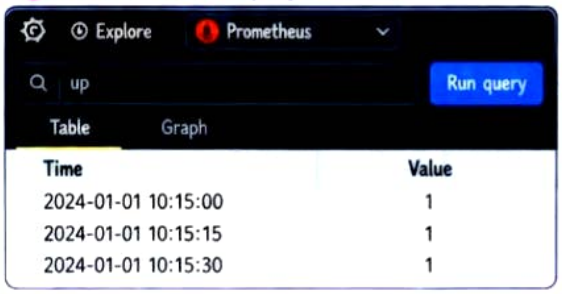
1. Login to Grafana (http://<server-ip>:3000)
2. Go to Configuration → Data Sources
3. Click Add data source
4. Select Prometheus
5. Enter Prometheus server URL (e.g. http://<ip>:9090)
6. Click Save & Test





7. Test Query in Grafana

1. Go to Explore in Grafana
2. Select Prometheus as data source
3. Write a PromQL query and run it



8. Common PromQL Queries

Query	Description
up	Check if targets are up or down
node_cpu_usage_seconds_total	CPU usage metrics
node_memory_MemAvailable_bytes	Available memory
rate(http_requests_total[5m])	HTTP request rate (last 5 minutes)
sum(rate(container_cpu_usage_seconds_total[5m]))	Total container CPU usage
kube_pod_status_phase	Pod status information

9. Create Dashboard in Grafana

- Go to Dashboards → New → New dashboard
- Add a panel and select Prometheus as data source
- Write a PromQL query (e.g. node_cpu_usage_seconds_total)
- Choose visualization type (Graph, Gauge, Stat, Table)
- Customize panel title, unit, thresholds and legend
- Click Apply → Save dashboard

10. Next Action

- Install Prometheus and Grafana on server
- Configure and scrape your applications
- Create useful dashboards (CPU, Memory, Pods, etc.)
- Set up alerts using Grafana (optional)

Grafana Dashboards & Panels

Create, Configure and Visualize Metrics using Dashboards and Panels

1. What are Dashboards & Panels ?

- A Dashboard in Grafana is a collection of panels to visualize metrics, logs and data.
- A Panel is an individual visualization (graph, table, gauge, etc.) that displays data from a data source.
- We can create multiple dashboards for different use cases (infrastructure, application, Kubernetes, AWS, etc.).
- Dashboards help in real-time monitoring, troubleshooting and better visibility.

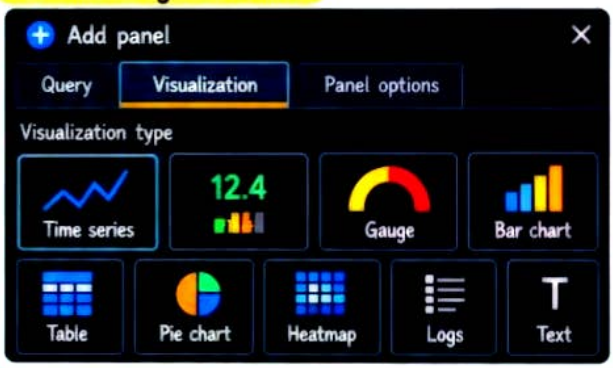
2. Dashboard Components



3. Steps to Create a Dashboard

1. Login to Grafana (http://<server-ip>:3000)
2. Go to Dashboards → New → New dashboard.
3. Click on Add panel.
4. Select data source (e.g. Prometheus).
5. Write a query (e.g. node_cpu_usage_seconds_total).
6. Choose visualization type (Graph, Gauge, Table, etc.).
7. Configure panel settings (title, unit, thresholds, legend).
8. Click Apply and Save dashboard.

4. Adding a Panel



5. Common Visualization Types

Visualization Type	Icon	Use Case
Time series (Graph)		Plot metrics over time (CPU, Memory, etc.)
Stat	123	Show a single value (e.g. total count)
Gauge		Show current value with threshold
Bar chart		Compare multiple values
Table		Display data in tabular format
Pie chart		Show percentage distribution
Heatmap		Visualize metric density (e.g. latency)
Logs		Display logs (Loki)
Text	T	Show text, links or custom information

6. Panel Configuration Options

Option	Description
Title	Set panel title
Data source	Select data source (Prometheus, CloudWatch, etc.)
Query	Write query to fetch data
Visualization	Choose graph, table, gauge, etc.
Unit	Set unit (%, Bytes, ms, etc.)
Min / Max	Set minimum and maximum values
Thresholds	Define alert thresholds with colors
Legend	Show/hide legend and customize
Transformations	Modify data (filter, rename, etc.)
Panel links	Add links to other dashboards or URLs
Repeat panel	Create multiple panels using variables

7. Example Panel (CPU Usage from Prometheus)



8. Dashboard Variables (for Dynamic Filtering)

- Variables allow you to create dynamic and reusable dashboards.
- Common use cases: filter by cluster, namespace, pod, instance, etc.
- Steps:
  1. Go to Dashboard settings → Variables.
  2. Click Add variable.
  3. Select data source and write query (e.g. label_values(kube_pod_info, namespace)).
  4. Use the variable in your panel query (e.g. $namespace).

9. Create Dashboard in Grafana

- Go to Dashboards → New → New dashboard
- Add panels and select Prometheus as data source
- Write PromQL queries (e.g. node_cpu_usage_seconds_total)
- Choose visualization type
- Configure panel settings
- Click Apply → Save dashboard

10. Next Action

- Create dashboards for infrastructure, applications, Kubernetes
- Set up variables for dynamic filtering
- Add alerts and thresholds
- Share dashboard with team

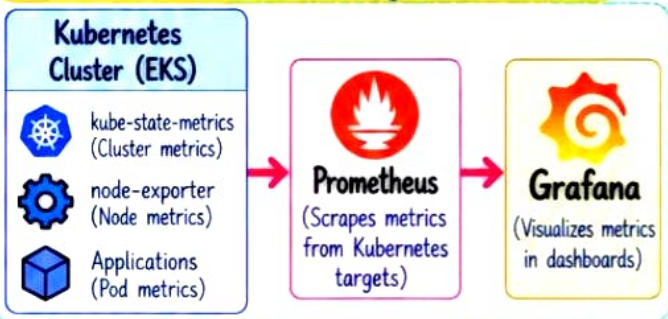
Kubernetes Monitoring with Grafana

Monitor Pods, Nodes, Containers and Cluster Metrics in Real-Time

1. Overview

- Grafana is used to visualize Kubernetes metrics using Prometheus as a data source.
- We use kube-state-metrics and node-exporter to collect Kubernetes cluster metrics.
- It helps to monitor Nodes, Pods, Containers, CPU, Memory, Network and more in real-time.
- Useful for troubleshooting, performance analysis and alerting in production environments.

2. Kubernetes Monitoring Architecture



3. Required Components

Component		Description	Installation
Prometheus		Collects and stores Kubernetes metrics	Helm / YAML
kube-state-metrics		Collects cluster object metrics (pods, deployments, etc.)	Helm / YAML
node-exporter		Collects node-level metrics (CPU, memory, etc.)	Helm / YAML
Grafana		Visualizes metrics in dashboards	Helm / Manual

4. Install kube-state-metrics (Helm)

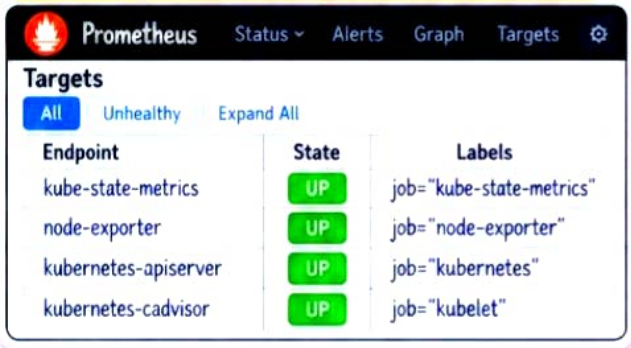
```
# Add Helm repo
helm repo add prometheus-community \
https://prometheus-community.github.io/helm-charts
helm repo update

# Install kube-state-metrics
helm install kube-state-metrics \
  prometheus-community/kube-state-metrics \
  --namespace monitoring \
  --create-namespace
```

5. Install node-exporter (Helm)

```
# Install node-exporter
helm install node-exporter \
  prometheus-community/prometheus-node-exporter \
  --namespace monitoring \
  --create-namespace
```

6. Verify Kubernetes Metrics in Prometheus



7. Add Prometheus Data Source in Grafana

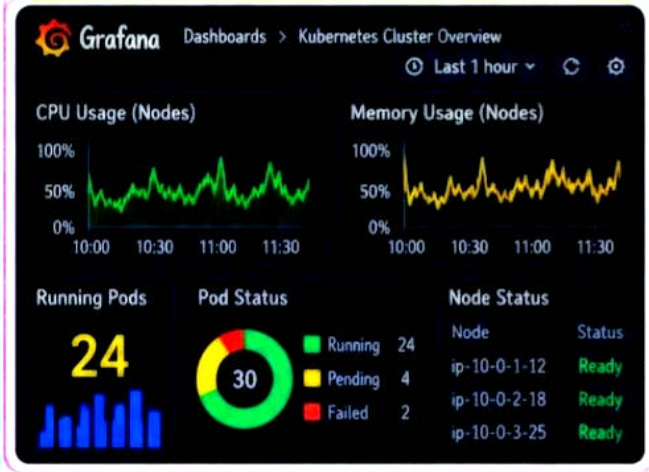
1. Login to Grafana (http://<server-ip>:3000)
2. Go to Configuration → Data Sources
3. Click Add data source
4. Select Prometheus
5. Enter Prometheus server URL (e.g. http://prometheus:9090)
6. Click Save & Test



8. Sample Kubernetes Metrics (PromQL Queries)

Query	Description
kube_pod_status_phase	Pod status (Running, Pending, etc.)
kube_deployment_status_replicas	Deployment replicas status
node_cpu_usage_seconds_total	Node CPU usage
node_memory_MemAvailable_bytes	Node available memory
container_cpu_usage_seconds_total	Container CPU usage
container_memory_usage_bytes	Container memory usage
kube_node_status_condition	Node ready status

9. Sample Grafana Dashboard (Kubernetes Overview)



10. Useful Dashboards

- Kubernetes / Compute Resources / Cluster (Grafana official dashboard)
- Node Exporter Full (system metrics)
- Kubernetes Pods / Deployments / Services
- Container resource usage
- Network, Disk, Memory dashboards

11. Key Monitoring Metrics

- Cluster health (Nodes, Pods, Deployments)
- CPU, Memory, Disk and Network usage
- Pod and Container resource utilization
- Application metrics (via Prometheus exporters)
- Alerts for high CPU, memory, pod failures, etc.

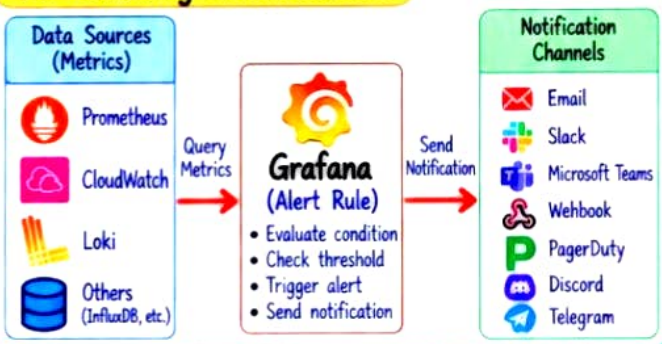
Grafana Alerts & Alert Rules

Set Alerts for Metrics, Configure Notifications and Handle Incidents

1. What are Grafana Alerts ?

- Grafana Alerts (Unified Alerting) allows you to monitor metrics and send notifications when a condition is met.
- Alerts help in early detection of issues like high CPU, high memory, pod down, application errors, etc.
- We can create alert rules based on data from Prometheus, CloudWatch, Loki and other data sources.

2. Alerting Architecture



3. Types of Alerts in Grafana

Alert Type	Description	Use Case
Threshold Alert	Triggers when a metric crosses a threshold	CPU > 80% Memory > 80%
No Data Alert	Triggers when no data is received	Metrics not coming from Prometheus
Error Alert	Triggers when a query returns an error	Data source issue
Change Alert	Triggers on sudden change in value	Spike in traffic
Multi-condition Alert	Use multiple conditions (AND / OR)	High CPU AND High Memory

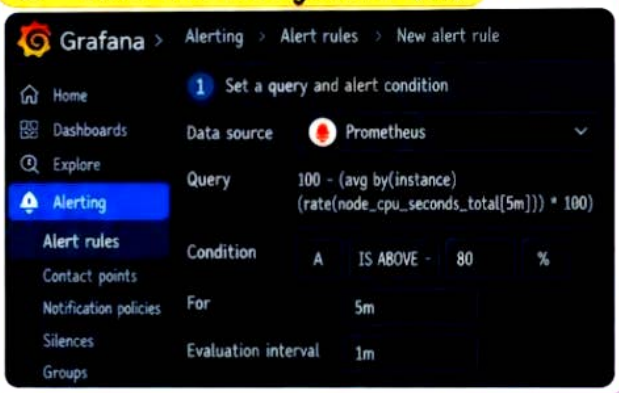
4. Create an Alert Rule (Steps)

1. Go to Alerting → Alert rules → New alert rule
2. Select data source (e.g. Prometheus)
3. Write a query (e.g. node_cpu_usage_seconds_total)
4. Set condition and threshold (e.g. > 80%)
5. Set evaluation interval and for duration
6. Configure notification channel (Email, Slack, etc.)
7. Click Save and Test

5. Example: CPU Usage Alert (Prometheus)

```yaml
apiVersion: 1
groups:
  - name: NodeCPUHigh
    interval: 1m
    rules:
      - alert: HighCPUUsage
        expr: 100 - (avg by(instance)
        (rate(node_cpu_seconds_total[5m])) * 100) > 80
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "High CPU usage on {{ $labels.instance }}"
          description: "CPU usage is above 80% for 5 minutes"
```

6. Alert Rule Configuration (UI)



7. Notification Channels

Channel	Icon	Description	Use Case
Email		Send alert via email	Basic alerts
Slack		Send alert to Slack channel	Team notifications
Microsoft Teams		Send alert to Teams	Enterprise teams
Webhook		Send HTTP request	Custom integration
PagerDuty		Incident management	Critical alerts
Discord		Send alert to Discord	Community / Team
Telegram		Send alert to Telegram	Personal alerts

8. Sample Alert Notification (Email)

From: grafana-alerts@company.com
To: devops-team@company.com
Subject: [ALERT] High CPU Usage on ip-10-0-1-12

High CPU Usage Alert

Alert: NodeCPUHigh
Instance: ip-10-0-1-12:9100
Severity: critical
Value: 85.3 %
Started at: 2024-01-01 10:25:00
Dashboard: http://grafana-ip:3000/d/abc123/server-metrics

View in Grafana

9. Common Alert Rules with Queries

Use Case	PromQL Query	Threshold
High CPU Usage	100 - avg(rate(node_cpu_seconds_total [5m])) * 100	> 80%
High Memory Usage	(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100	> 80%
Pod Down	kube_pod_status_phase != 1	> 0
High Network Traffic	rate(container_network_receive_bytes_total[5m])	> 100MB/s
Disk Usage	node_filesystem_avail_bytes / node_filesystem_size_bytes * 100	> 80%

10. Best Practices (Production)

- Set meaningful thresholds (avoid false alerts)
- Use for duration (e.g. 5m) to avoid flapping
- Add proper labels and annotations
- Group related alerts
- Use multi-condition rules for critical services
- Configure multiple notification channels
- Test alerts before using in production
- Monitor and tune alerts regularly

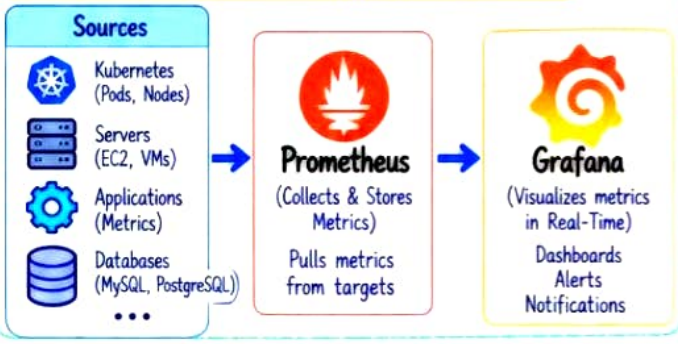
Real-Time Production Monitoring

Monitor Infrastructure, Applications and Business Metrics in Real-Time using Grafana

1. What is Real-Time Monitoring ?

- Real-time monitoring means collecting, processing and visualizing metrics, logs and events as they happen in the production environment.
- Grafana (visualization) + Prometheus (metrics) helps us to monitor system health, application performance and business metrics in real-time.
- It helps in early detection of issues, faster troubleshooting and ensures high availability of applications.

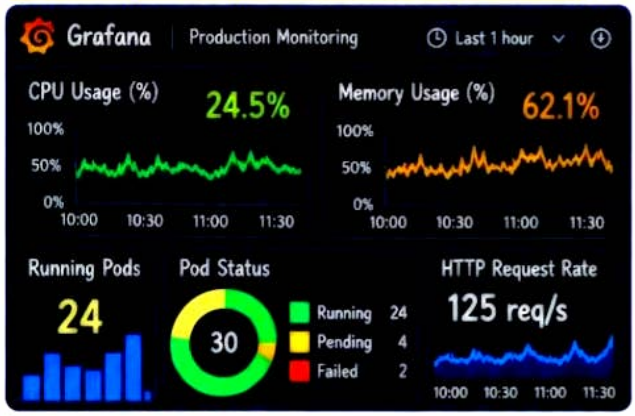
2. Production Monitoring Architecture



3. Key Metrics to Monitor in Production

Category	Key Metrics	Use Case
Infrastructure	CPU, Memory, Disk, Network, Load	Monitor server health and capacity
Kubernetes	Node status, Pod status, Pod restarts, Resource usage	Ensure cluster and pod health
Application	Response time, Error rate, Request count	Monitor application performance
Database	Connections, Query time, CPU, Memory	Ensure database availability
Business	Orders, Transactions, Revenue, Queue size	Track business KPIs in real-time

4. Sample Grafana Dashboard (Production)



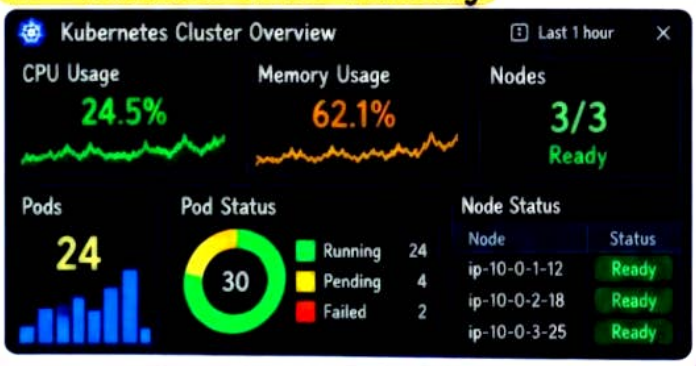
5. Useful PromQL Queries

Query	Description
rate(node_cpu_usage_seconds_total[5m]) * 100	CPU usage (%)
node_memory_MemAvailable_bytes	Available memory
sum(kube_pod_status_phase{phase="Running"})	Running pods count
sum(kube_pod_container_status_restarts_total)	Pod restart count
rate(http_requests_total[5m])	HTTP request rate
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))	95th percentile response time

6. Application Monitoring Dashboard



7. Kubernetes Cluster Monitoring



8. Infra Monitoring (EC2 / Server)



9. Database Monitoring (MySQL / PostgreSQL)



10. Alerting in Production

- Set alerts for high CPU, high memory, pod down, error rate, etc.
- Configure alert rules based on thresholds
- Use notification channels (Email, Slack, Teams, Webhook, PagerDuty)
- Helps in faster incident response and reduces downtime

11. Real-World Use Cases

- Monitor Kubernetes / Compute Resources / Cluster (Grafana official)
- Monitor application performance (APIs, Microservices)
- Monitor databases (MySQL, PostgreSQL)
- Track business metrics (Orders, Revenue, Queue size)
- Set up alerts and notifications for incidents
- Used in DevOps, SRE and production environments

Real-World DevOps Grafana Project

Kubernetes → Prometheus → Grafana → Dashboard → Alert → Incident → Root Cause → Resolution

1. Project Objective

- Monitor a Kubernetes application in real-time using Prometheus and Grafana.
- Create dashboards for cluster and application metrics.
- Set up alerts for critical metrics (CPU, Memory, Pod Down, etc.).
- Generate an alert, investigate the incident, find the root cause and apply the fix.

2. High Level Architecture

Kubernetes (EKS / Cluster)
- Pods
- Nodes
- Applications
- kube-state-metrics
- node-exporter

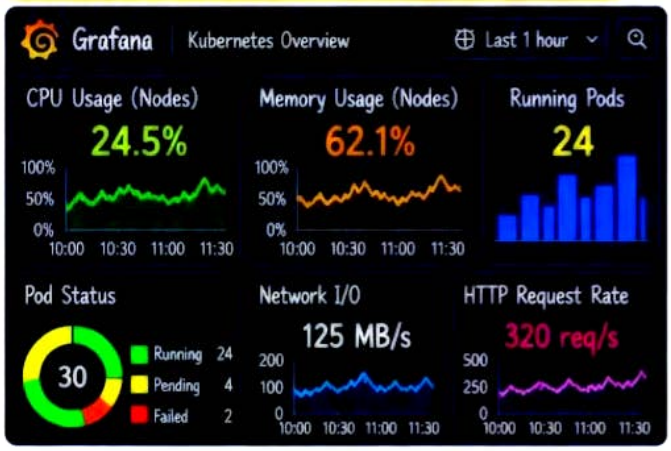
→ **Prometheus**
- Scrapes metrics
- Stores time-series data
- Evaluates alert rules

→ **Grafana**
- Create dashboards
- Visualize metrics
- Configure alerts
- Send notifications

3. Step by Step Implementation

1. Deploy a sample application on Kubernetes (e.g. Nginx or online boutique).
2. Install Prometheus (using Helm) with kube-state-metrics and node-exporter.
3. Configure Prometheus to scrape Kubernetes metrics.
4. Install Grafana and add Prometheus as data source.
5. Create dashboards for cluster and application metrics.
6. Set up alert rules (e.g. high CPU, pod down).
7. Generate an alert and simulate an incident.
8. Investigate logs/metrics, find root cause and fix it.

4. Grafana Dashboard (Kubernetes Overview)



5. Sample Alert Rule (High CPU)

```yaml
groups:
- name: kubernetes-alerts
  rules:
  - alert: NodeCPUHigh
    expr: 100 - (avg by (instance)
    (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
    for: 5m
    labels:
      severity: critical
      service: kubernetes
    annotations:
      summary: "High CPU usage on {{ $labels.instance }}"
      description: "CPU usage is above 80% for more than 5 minutes."
```

6. Alert Notification (Email / Slack)

From: grafana-alerts@company.com
To: devops-team@company.com
Subject: [ALERT] High CPU Usage on ip-10-0-1-12

High CPU Usage Alert

Alert:	NodeCPUHigh
Instance:	ip-10-0-1-12:9100
Severity:	critical
Value:	85.3 %
Started at:	2026-10-01 10:25:00
Description:	CPU usage is above 80% for more than 5 minutes.

View in Grafana

7. Incident (What Happened?)

At 10:25 AM we received an alert:
- High CPU usage on node ip-10-0-1-12.
- Application response time increased.
- Few pods were running slow.
- Users started facing issues in the application.

8. Root Cause Analysis

1. Checked Grafana dashboard → CPU usage was at 85%.
2. Opened Prometheus graph to verify the metric trend.
3. Checked Kubernetes top → one pod had high CPU.
4. Checked application logs → found a specific process consuming high CPU (infinite loop / high traffic).
5. Verified recent deployment → new version caused a CPU spike.

9. Resolution Steps

- Rolled back the deployment to the previous stable version.
- Restarted the affected pod.
- Verified CPU usage came back to normal (< 50%).
- Confirmed application performance was restored.
- Kept the alert active for future monitoring.
- Added a note in incident management tool (PagerDuty / Jira).

10. Final Architecture & Flow (End to End)

Application (Deployment) → Kubernetes (Pods, Nodes) → kube-state-metrics & node-exporter → Prometheus (Metrics + Alerting) → Grafana (Dashboard + Alerts) → Notifications (Email / Slack) → Incident (Root Cause & Resolution)